

The Power of Minimalism in Deep Learning

A walkthrough of **VanillaNet** — a network that throws out depth, shortcuts and self-attention, yet matches ResNet and Swin Transformer. How? By moving complexity out of *inference* and into *training*.

6–13

layers total — no shortcuts,
no attention

79.9%

ImageNet top-1 at 9 layers

>50%

faster than ResNet-50 at
equal accuracy

Chen, Wang, Guo, Tao — **Huawei Noah's Ark Lab** & University of Sydney · arXiv:2305.12972 (2023)

Press → or Space to begin.

“More is different” built modern AI. This paper asks: **what if less is enough?**

Foundation models grew by piling on depth and intricate operators. VanillaNet bets the opposite: a network so plain it's almost trivial to deploy — and proves the plainness need not cost accuracy.

PART 1 · AUTHOR INTENT

Why minimalism?

The problem, the contribution, and how it breaks from ResNet-era thinking.

PART 2 · KEY CONCEPTS

The two tricks

Deep-training collapse & series-informed activation — where the magic hides.

PART 3 · IMPLEMENTATION

It actually works

The architecture, the merge math, ablations and ImageNet / COCO results.

Keep one question in mind: if a layer adds no accuracy at inference, why pay for it at inference? VanillaNet's answer is to **use** it while training and **delete** it before deployment.

Complexity is great for accuracy — and terrible for deployment.

Every accuracy win of the last decade came bundled with an engineering tax that shows up at inference time:



Shortcuts cost memory traffic

ResNet's skip connections merge features from different layers, consuming heavy off-chip memory bandwidth on real hardware.



Exotic ops need custom kernels

Shifted-window attention (Swin) and axial-shift (AS-MLP) require hand-written CUDA — fragile to port and deploy.



Depth blocks parallelism

ViT-Base is ~62 layers; each must finish before the next starts. Sequential depth can't be parallelised away.



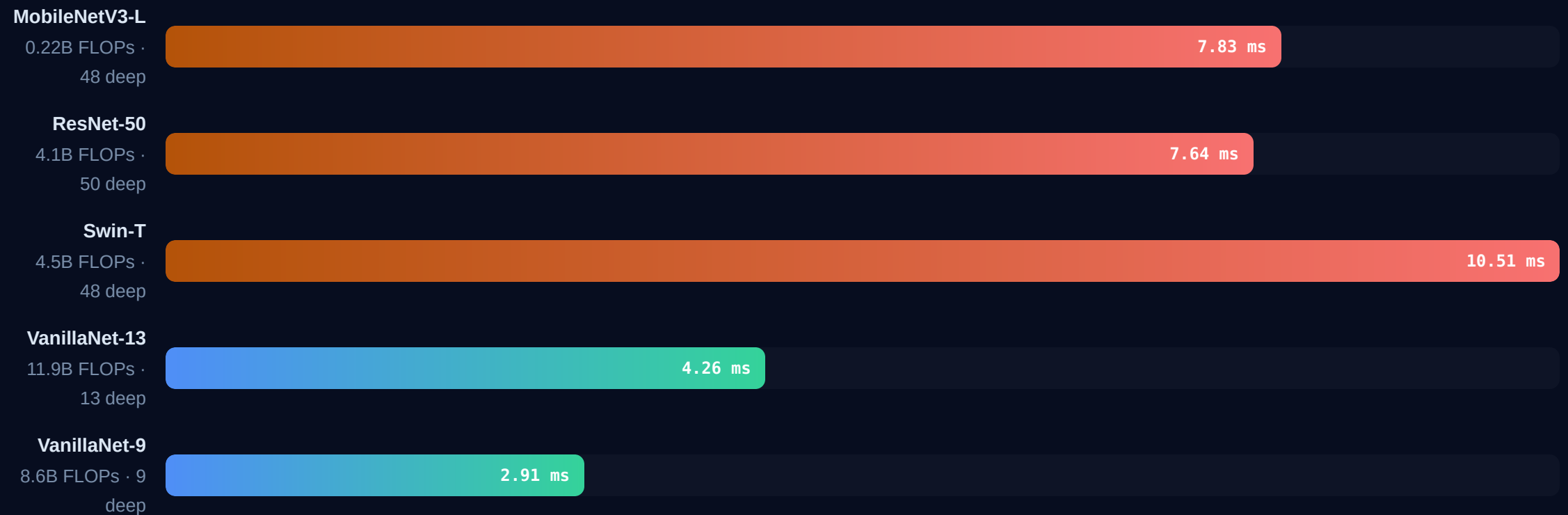
Plain nets were abandoned

Since ResNet, pure-conv nets fell out of fashion: a 34-layer plain net trained *worse* than an 18-layer one (gradient vanishing).

The gap: nobody was optimising **simple, shallow** networks anymore — even though they're by far the easiest to ship. VanillaNet reopens that abandoned door.

On a modern GPU, **depth** — not FLOPs — sets the speed.

With batch size 1 and abundant compute, a GPU parallelises within a layer effortlessly. What it *can't* hide is the sequential chain of layers. So a “cheap” deep net can be slower than an “expensive” shallow one:



GPU latency, batch size 1, Nvidia A100. VanillaNet-13 has **54×** the FLOPs of MobileNetV3-L yet runs **~45% faster**.

The reframing: if latency is bounded by depth, the winning move isn't fewer FLOPs — it's **fewer layers**. Spend FLOPs freely inside each layer; just don't stack many.

A deliberate U-turn in the architecture story.



2012 · AlexNet

Shallow & plain wins ImageNet

12 plain layers beat everything. The starting point — simple, stackable conv blocks.



2014 · VGGNet

Go deeper, stay plain

Uniform 3×3 stacks. Depth helps... until plain nets stop training well.



2015 · ResNet — the fork

Shortcuts rescue depth

Skip connections fix gradient vanishing, so 50–1000 layers become trainable. **But the field reads this as “plain nets are dead.”**



2020–22 · ViT · Swin · ConvNeXt

Even deeper, even more intricate

Self-attention and elaborate blocks push accuracy higher — and deployment complexity with it.



2023 · VanillaNet

Revisit the fork — go shallow, not deep

Take the abandoned plain-net path, but instead of fighting gradient vanishing with depth, **avoid depth entirely** and fix the real bottleneck: weak non-linearity.

Stem → body → head. One layer per stage.

The classic recipe — but stripped to the bone. Each stage is a single 1×1 conv + activation + max-pool. Channels double, resolution halves. No branches, no SE-blocks, **no shortcuts** (they tested — shortcuts didn't help).



■ Stem: 4×4 conv, stride 4 — RGB → C channels ■ Body: 1×1 convs + BN + activation + maxpool ■ Head: average pool → fully-connected

Why this is a deployment dream: every layer is a plain 1×1 conv — a single matrix multiply. No CUDA tricks, no merged branches, runs fast on literally any hardware.

Shallow + plain = **weak non-linearity** = weak accuracy.

A naive 6-layer plain ReLU net tops out around **60.5%** on ImageNet — useless in practice. The bottleneck isn't identity mapping (shortcuts don't help). It's that too few activation layers means too little non-linearity.

The whole paper is two ideas that **inject non-linearity** — without adding a single layer or operation to the deployed model:

Trick 1 · Deep Training

Train as a *deeper* net, then mathematically collapse the extra layers into the shallow one before inference. Capacity while learning, simplicity while serving.

train-time only

zero inference cost

Trick 2 · Series Activation

Make each activation far more expressive by stacking it *in parallel*, so one activation layer does the work of many.

richer per-layer

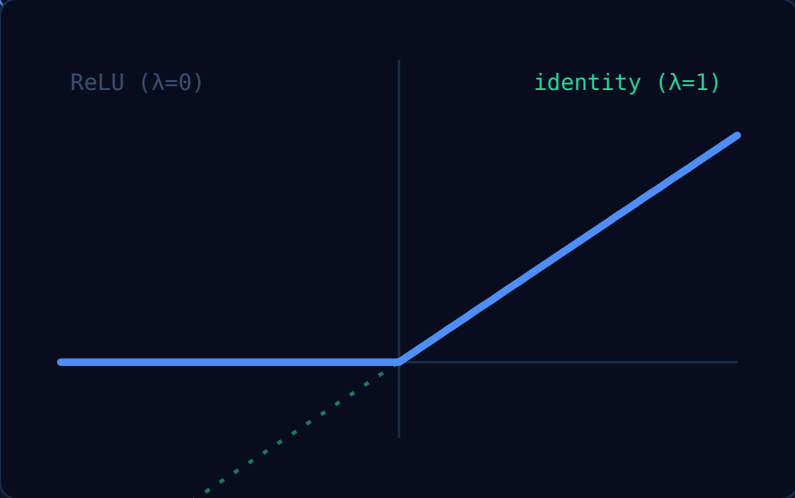
~1% of conv FLOPs

Start deep with activations. Fade them to identity. Merge.

Each layer begins as **two** convs with an activation between them. The activation is blended toward the identity over training: $A'(x) = (1-\lambda) \cdot A(x) + \lambda \cdot x$ with $\lambda = e / E$. Drag λ to watch it collapse:

Training progress $\lambda = \text{epoch} / \text{total}$

0.00



ReLU ($\lambda=0$)

identity ($\lambda=1$)

Blue = $A'(x)$ at $\lambda=0.00$, blended between ReLU and the dashed identity line.

Conv₁
1×1

→

A'(x)
active

→

Conv₂
1×1

Effective depth while training: 2× convs.

Non-linearity retained: 100%.

Σ The merge: two layers become one, exactly & for free

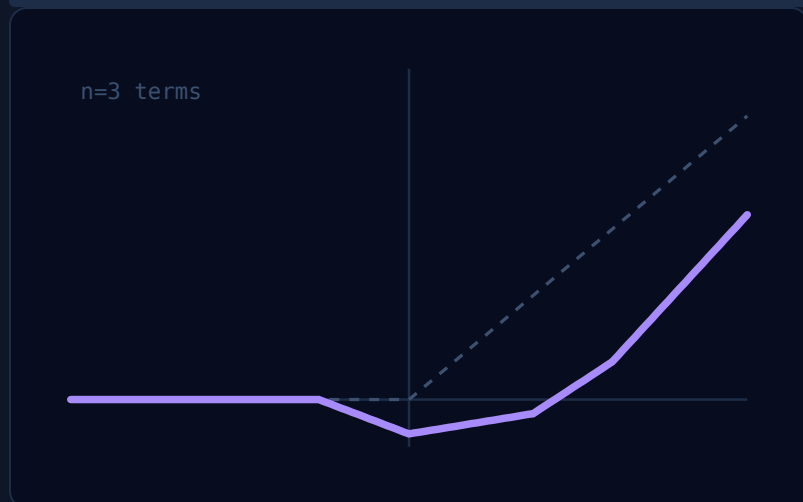
show math ▼

Stack activations **in parallel**, not in series.

Deep nets gain non-linearity by stacking activations *serially* (= more depth = more latency). VanillaNet stacks them *concurrently* inside one layer: a learnable weighted sum of shifted ReLUs that approximates a far richer curve. Drag **n**:

Terms in the series **n**

3



$n=0$ is plain ReLU. Each added term is a scaled, shifted ReLU — together they bend into smooth, expressive shapes.

$$As(x) = \sum_{i=1}^n a_i \cdot A(x + b_i)$$

a_i, b_i are learned per channel. Extended to also mix spatial neighbours (like BNET), so one activation gathers **global** context.

Series terms: **3**

ImageNet top-1: **76.36%** ← paper default

Ablation: $n=0 \rightarrow 60.5\%$ · $n=1 \rightarrow 74.5\%$ · $n=3 \rightarrow 76.4\%$. One idea recovers nearly all the lost accuracy.

Σ Why it's almost free: the complexity ratio

show math ▼

Both tricks are essential. Shortcuts are not.

VanillaNet-6 · stacking the two ideas

DEEP TRAIN	SERIES ACT	TOP-1
—	—	59.58%
✓	—	60.53%
—	✓	75.23%
✓	✓	76.36%

Series activation does the heavy lifting; deep training adds a further point. Together: **+16.8%**.

Do shortcuts help a shallow net?

SHORTCUT TYPE	TOP-1
No shortcut	76.36%
Before activation	75.92%
After activation	75.72%

The key realisation: shortcuts *hurt* slightly — they skip activations, lowering non-linearity. So the vanilla-net bottleneck was never identity mapping; it was **non-linearity**. That’s why removing shortcuts is safe.

Applied to AlexNet, the same two tricks add ~6%. Applied to ResNet-50, almost nothing — deep nets already have enough non-linearity. The tricks specifically cure **shallow** nets.

Matches the giants — at a fraction of the latency.

MODEL	DEPTH	PARAMS	FLOPS	LATENCY	TOP-1
ResNet-50-TNR	50	25.6M	4.1B	7.64 ms	79.8%
ConvNeXtV2-P	41	9.1M	1.37B	6.29 ms	79.7%
Swin-S	96	49.6M	8.7B	20.25 ms	83.2%
VanillaNet-9	9	41.4M	8.6B	2.91 ms	79.9%
VanillaNet-13-1.5x†	13	127.8M	48.9B	9.72 ms	83.1%

2.6×

faster than ResNet-50 at the same 79.9% accuracy (VanillaNet-9)

83.1%

top-1 when scaled up — into Swin-S territory at 13 layers

+4.4

FPS on COCO Mask-RCNN vs Swin-T backbone — the speed edge transfers downstream

A wide, holistic gaze — its strength **and** its limit.

Grad-CAM++ on VanillaNet-9 vs ResNet-50 reveals a different style of seeing:

Strength · broad coverage

VanillaNet activates over a **much larger** region. With many channels and FLOPs per layer, it extracts *all* relevant signal across the image rather than a narrow patch.

Limit · weaker isolation

Its lower non-linearity makes it **less able to focus** on a single discriminative part — the honest trade-off of going shallow.

Interpretation: depth buys the ability to carve out fine, part-specific attention. VanillaNet compensates with width and brute coverage — good enough to match deep nets on classification, by a different mechanism.

Two models in one codebase.

The released implementation cleanly separates the **training graph** from the **deployment graph**:



Training mode

Each block = Conv $\rightarrow$ BN $\rightarrow$ **A'(x)** $\rightarrow$ Conv $\rightarrow$ BN, with series activation. λ scheduled **0** $\rightarrow$ **1** over the deep-training epochs. Strong regularisation (large per-layer capacity).



Deploy mode · `switch_to_deploy()`

Fold BN $\rightarrow$ conv, then fold the two convs into **one**. Activations are now identity, so they disappear. Result: a flat stack of single 1 $\times$ 1 convs.

STEP 1

Train deep

2 convs + active A' per stage.

STEP 2

Anneal $\lambda \rightarrow 1$

Activations fade to identity.

STEP 3

Fold BN

Eq. 2 — BN into conv.

STEP 4

Merge convs

$W^1 \cdot W^2 \rightarrow$ one layer. Ship it.

What VanillaNet really teaches.



Latency $\neq$ FLOPs

When compute is abundant, **depth** is the cost. Optimise the metric that actually bottlenecks deployment.



Decouple train & infer

Capacity is only needed while learning. **Collapse** it away before serving — re-parameterisation as a design principle.



Non-linearity > depth

For shallow nets the real bottleneck is per-layer non-linearity, not identity mapping. Fix **that**, drop the shortcuts.

The minimalist thesis, proven: you can match ResNet and Swin with 6–13 branch-free layers — by moving every drop of complexity into training and deleting it before inference. **Less, done cleverly, is enough.**

Limitations worth raising in Q&A: high params/FLOPs per layer (memory-heavy), weaker part-level focus, strong-regularisation dependence, and classification-centric evaluation.

THANK YOU

Questions?

VanillaNet · the Power of Minimalism in Deep Learning — Chen, Wang, Guo & Tao,
Huawei Noah's Ark Lab.

[arXiv:2305.12972](#)

[github.com/huawei-noah/VanillaNet](#)

[Deep-training collapse](#)

[Series-informed activation](#)

Use [←](#) / [→](#) to revisit any section · press [F](#) for fullscreen.